



Algorithms and Complexity 2026

Exercise Session 4: Flows and Dynamic Programming

This is a collection of problems that hopefully improve your understanding of the material discussed in the lectures. During the exercise classes you have time to work on the first 3-5 problems and will have the opportunity to obtain feedback and/or help from one of the teaching assistants. Towards the end of each session the solutions of these problems will be covered.

To optimally prepare for the mastery test, you are encouraged to solve some of the remaining problems on your own. If you are stuck, require assistance or someone really messed up to write an understandable solution, feel free to send an email to algokomp-help@kth.se or visit an office hour.

Some problems are marked with superscripts: 1^M , 2^* or even 3^{**} . Problems marked with an **M** superscript are in the spirit of mastery test problems. Such problems are usually not stated as plain math problems but rather consist of a story from which you need to extract the mathematical structure yourself. Problems marked with a $\star$ are challenging problems. The more stars the harder – do not get discouraged if you cannot solve such a problem.

1 Edmonds-Karp

Consider the graph in Figure 1. Run the Edmonds-Karp algorithm on this graph, given that node 1 is the source and node 8 is the sink. Let us assume that we remove vertices from the queue in increasing order of the label of the vertices. What is the maximum flow from node 1 to 8?

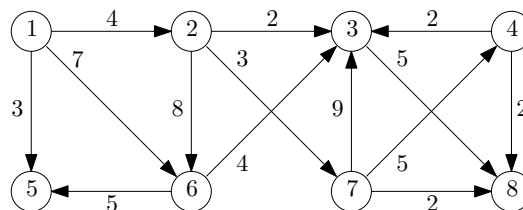


Figure 1: The number incident to an edge indicates its capacity.

Solution:

The following figure indicates the updated flow after finding a path from source to sink.

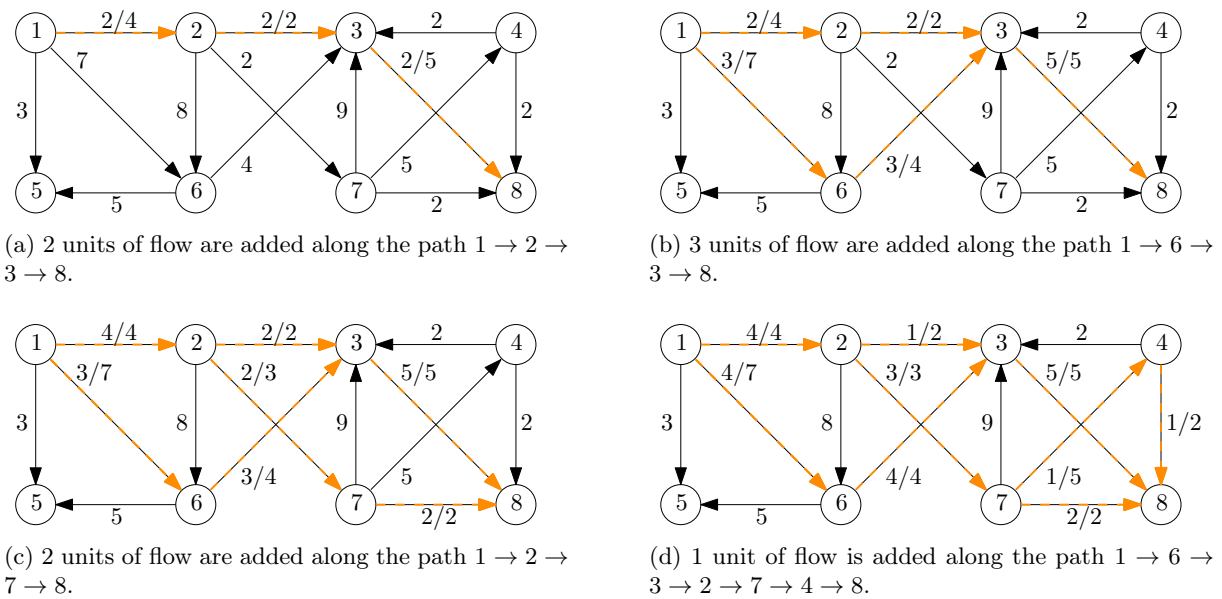


Figure 2: In the pairs i/j incident to edges, i is the current flow while j is the capacity of the edge. We only write the flow if it is non-zero.

2 Arithmetic Sum

Consider [Algorithm 1](#) which adds up the integers from 1 to n . You may assume that adding to ℓ -bit numbers is of time complexity $\mathcal{O}(\ell)$ with regards to the bit-cost model.

Algorithm 1 sums all integers from 1 to n .

```

1: function SUM( $n$ )
2:    $s \leftarrow 0$ 
3:   for  $i \leftarrow 1, 2, \dots, n$  do
4:      $s \leftarrow s + i$ 
5:   return  $s$ 

```

- Prove by induction that $\sum_{i=1}^n i = \frac{n(n+1)}{2}$.
- Analyze the time complexity of [Algorithm 1](#) in the bit- and unit-cost model. Express your answer as a function of n .
- Note that the input to SUM is only of size $\mathcal{O}(\log n)$: it requires $\mathcal{O}(\log n)$ bits to express a number of magnitude n .
Analyze the time complexity of [Algorithm 1](#) in the bit and unit-cost model. Express your answer as a function of the size of the input. You may assume that the input is of size $m = \log n$.
- Using (a), come up with an algorithm of time complexity $\mathcal{O}(1)$ in the unit-cost model.
- Analyze the new algorithm in the bit-cost model. Express the time complexity once with respect to n and once with respect to m . You may assume that the multiplication of two ℓ -bit numbers is of time complexity $\mathcal{O}(\ell^2)$.

Solution:

(a) Let's prove the result by induction.

◇ *Induction Hypothesis:* For $n \in \mathbb{N}^*$, let H_n : " $\sum_{i=1}^n i = \frac{n(n+1)}{2}$ ".

◇ *Base Case:* Let's prove that H_1 holds.

$$\sum_{i=1}^1 i = 1 = \frac{2}{2}$$

Thus, H_1 holds.

◇ *Induction Step:* Let's suppose that H_n holds for some $n \in \mathbb{N}^*$, and let's show that H_{n+1} holds.

By induction hypothesis, we know that $\sum_{i=1}^n i = \frac{n(n+1)}{2}$. Then, we can compute:

$$\begin{aligned} \sum_{i=1}^{n+1} i &= (n+1) + \sum_{i=1}^n i \\ &= (n+1) + \frac{n(n+1)}{2} && \text{by induction hypothesis} \\ &= \frac{(2n+2) + n(n+1)}{2} \\ &= \frac{n^2 + 2n + n + 2}{2} \\ &= \frac{(n+1)(n+2)}{2} \end{aligned}$$

Thus, H_{n+1} holds.

◇ *Conclusion:* We showed that H_1 holds, and if H_n holds for some $n \in \mathbb{N}^*$, then H_{n+1} holds. Thus, by induction, we can conclude that for all $n \in \mathbb{N}^*$, H_n holds.

(b) Note that the for-loop is executed n times, doing a constant operation each time (an addition). Hence in the unit-cost model the time complexity is $\mathcal{O}(n)$.

From the previous task we know that s consists of at most $\mathcal{O}(\log n)$ bits. Thus each addition in the for-loop is of time complexity at most $\mathcal{O}(\log n)$. Thus the overall bit-cost time complexity is $\mathcal{O}(n \log n)$.

(c) Substituting $m = \log n$, we get that the unit-cost time complexity is $\mathcal{O}(2^m)$ and the bit-cost time complexity is $\mathcal{O}(m2^m)$.

(d) Compute $\frac{n(n+1)}{2}$. This only requires a multiplication, an addition and a division all of which we can perform in constant time in the unit-cost model.

(e) The addition is of time complexity $\mathcal{O}(\log n)$, while the multiplication is of time complexity $\mathcal{O}((\log n)^2)$. Note that a division by 2 can be done in linear time by shifting the bits one position to the right. Hence the time complexity in the bit-cost model is $\mathcal{O}((\log n)^2)$, or, expressed with respect to m , it is $\mathcal{O}(m^2)$.

Note that this is an exponential speedup compared to [Algorithm 1](#).

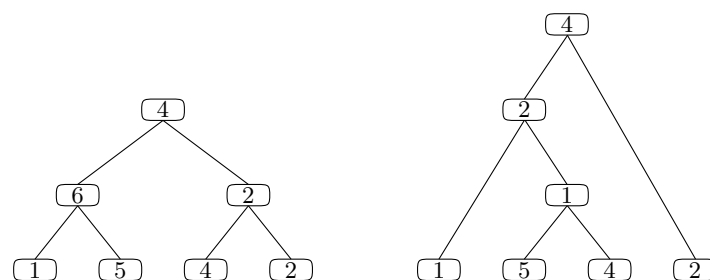
3^M Balanced Branching

You have a list of n integers $x_1, \dots, x_n$. For some inexplicable reason you would now like to build a binary tree on top of these numbers, with each leaf containing exactly one of the numbers. The numbers have to remain in the same order x_1 up to x_n .

We define the *discrepancy* of a node in the tree to be the absolute value of the sum of the values of the leaves below that are descendants of that node. We further define the discrepancy of the tree to be the sum of the discrepancies of all nodes in the tree.

Design an efficient (polynomial in n and independent of the x_i 's assuming the unit cost model) algorithm to find the minimum possible discrepancy of all possible trees on the input values $x_1, \dots, x_n$.

Example: suppose $n = 4$ and the values are $(-1, -5, 4, -2)$. Here are two possible trees that can be built on these numbers; the nodes are labelled by their discrepancy:



The tree on the left has a discrepancy of 24, and the one on the right has a discrepancy of 19, which is also the minimum possible discrepancy for this input.

Solution:

We will solve this problem using dynamic programming.

The problem is in fact very similar to the “Matrix Chain Multiplication” problem studied during Lecture 5: another way of thinking about the problem is that we are adding up the given numbers by repeatedly adding two consecutive numbers together (this corresponds to joining them together in the tree), and our goal is to make sure that the sum of the (absolute value of the) partial results encountered during the process is as small as possible.

We will use essentially the same solution as for the Matrix Chain Multiplication problem. Consider the root vertex of an optimal solution. Its left subtree must consist of the values $x_1, \dots, x_i$ for some i , and its right subtree must contain the remaining values $x_{i+1}, \dots, x_n$. In general, any subtree of an optimal solution contains the values $x_i, \dots, x_j$ for some indices $i \leq j$. So we will use such intervals as subproblems.

In particular, let us define $\text{Opt}(i, j)$ to be the minimum discrepancy of a tree containing the values $x_i, \dots, x_j$. Let us also define $\text{IvalDisc}(i, j) = \left| \sum_{k=i}^j x_k \right|$ to be the absolute value of the sum of the values x_i up to x_j . Then we have the recurrence

$$\text{Opt}(i, j) = \begin{cases} |x_i| & \text{if } i = j \\ \min_{i < k \leq j} (\text{Opt}(i, k-1) + \text{Opt}(k, j) + \text{IvalDisc}(i, j)) & \text{if } i < j \end{cases}$$

With this recurrence in place, here is pseudo code for a bottom-up dynamic programming implementation:

Algorithm 2

```
1: function IVALDISC( $i, j$ )
2:    $r \leftarrow 0$ 
3:   for  $k \leftarrow i, i + 1, \dots, j$  do
4:      $r \leftarrow r + x_k$ 
5:   return  $|r|$ 
6:  $A \leftarrow n \times n$  matrix indexed by indices  $1, 2, \dots, n$ 
7: for  $i \leftarrow 1, 2, \dots, n$  do
8:    $A[i, i] \leftarrow |x_i|$ 
9: for  $i \leftarrow n, n - 1, \dots, 1$  do
10:  for  $j \leftarrow i + 1, i + 2, \dots, n$  do
11:     $A[i, j] \leftarrow \infty$ 
12:     $d \leftarrow \text{IVALDISC}(i, j)$ 
13:    for  $k \leftarrow i + 1, i + 2, \dots, j$  do
14:       $t \leftarrow A[i, k - 1] + A[k, j] + d$ 
15:      if  $t < A[i, j]$  then
16:         $A[i, j] \leftarrow t$ 
```

The answer to the problem is given by the value $A[1, n]$.

Let us analyze the time complexity. There are $O(n^2)$ different inputs (i, j) . For each such input, we call $\text{IVALDISC}(i, j)$ which takes $O(j - i) = O(n)$ time, and then loop through $j - i = O(n)$ possible values of k . Overall the total work for input (i, j) is $O(n)$, and this leads to $O(n^3)$ total time.

4★ Tourist¹

A lazy tourist wants to visit as many interesting locations in a city as possible without going one step further than necessary. Starting from his hotel, located in the north-west corner of city, he intends to take a walk to the south-east corner of the city and then walk back. When walking to the south-east corner, he will only walk east or south, and when walking back to the north-west corner, he will only walk north or west. After studying the city map he realizes that the task is not so simple because some areas are blocked. Therefore he has kindly asked you to write a program to solve his problem.

Given the city map (a 2D grid) where the interesting locations and blocked areas are marked, determine the maximum number of interesting locations he can visit. Locations visited twice are only counted once.

The time complexity of your algorithm should be $O(n^{3.9})$.

Solution:

The idea is to view this as a problem where we create 2 walks from the north-west to the south-east corner. Let $L(s_1, s_2)$ be the maximum number of interesting locations that can be visited if the two walks start at s_1 and s_2 .

As the walks are done simultaneously, we are only interested in s_1, s_2 that are at equal distance from the north-west corner. Hence when we come up with the recurrence relation either we are currently at the same location or the two paths do not cross at this distance.

¹This problem is available on [Kattis](#).

If we view each location as a tuple (x, y) , where x is the north-south and y the east-west direction, we can write $L(s_1, s_2)$ as a recurrence relation in the following way

$$L((x, y), (x', y')) = 1_{\{(x, y) \text{ is interesting}\}} + 1_{\{(x', y') \text{ is interesting}\}} \quad (1)$$

$$- 1_{\{(x, y) = (x', y')\}} \cdot 1_{\{(x, y) \text{ is interesting}\}} \quad (2)$$

$$- \infty \cdot 1_{\{(x, y) \text{ is blocked or } (x', y') \text{ is blocked}\}} \quad (3)$$

$$+ \max_{b, b' \in \{0, 1\}} L((x + b, y + 1 - b), (x' + b', y' + 1 - b')), \quad (4)$$

where $1_{\{P\}}$ is the indicator function that evaluates to 1 if P is true and to 0 otherwise. In words, we check whether the current places are interesting and if they are the same, then we ensure that we only count this place once. If one of the current places is blocked we set the value to $-\infty$. Last we look for the best paths from where we could have come from.

Computing $L((0, 0), (0, 0))$ using memoization we obtain an algorithm of running time $O(n^4)$. But recall that we are only interested in the values $L(s_1, s_2)$ where s_1 and s_2 are at equal distance from the north-west corner. This shows that if 3 coordinates are chosen, then the 4th is determined. Hence the time complexity is in fact $O(n^3)$.

5★ Analyzer for context-free grammars

A context-free grammar is usually used to describe the syntax of a particular programming language. A context-free grammar in *Chomsky Normal Form* is described by

- a set of final symbols T (usually written in small letters),
- a set of non-final symbols N (usually written in capital letters),
- the initial symbol S (a non-final symbol in N),
- a set of rewrite rules of type $A \rightarrow BC$ or $A \rightarrow a$, for $A, B, C \in N$ and $a \in T$.

If $A \in N$ we define $\mathcal{L}(A)$ as

$$\mathcal{L}(A) = \{bc : b \in \mathcal{L}(B) \text{ and } c \in \mathcal{L}(C) \text{ where } A \rightarrow BC\} \cup \{a : A \rightarrow a\}.$$

The language generated by the grammar is then defined as $\mathcal{L}(S)$, ie. the set of all strings of the final symbols that can be formed by a rewriting chain from symbol S .

Example: Consider the grammar $T = \{a, b\}$, $N = \{S, A, B, R\}$, start symbol S and rules $S \rightarrow AR$, $S \rightarrow AB$, $A \rightarrow a$, $B \rightarrow b$, $R \rightarrow SB$. We can see that string $aabb$ is in the language generated by the grammar using the following chain of rewritings:

$$S \rightarrow AR \rightarrow aR \rightarrow aSB \rightarrow aSb \rightarrow aABb \rightarrow aaBb \rightarrow aabb.$$

In fact, one can show that the language generated by the grammar is the set of all strings consisting of k symbols a followed by k symbols b , where k is a positive integer.

Your task is to *design* and *analyze* an efficient algorithm that determines if a string is in the language generated by a given grammar. The input is a context-free grammar in Chomsky Normal Form, and a string of final symbols. The output is true or false depending on whether the string can be generated by the grammar or not. Calculate the time complexity of your algorithm in terms of number of rules m of the grammar and length n of the string.

Solution:

We use dynamic programming to solve this problem.

The input is a set of rules R and a word w , indexed from 1 to n . Let us build a $n \times n$ dimensional matrix M , where the element $M[i, j]$ is a set of non-final symbols from which one can derive the substring $w[i..j]$ from a chain of rewriting rules. We can define $M[i, j]$ recursively as

$$M[i, j] = \begin{cases} \{X : (X \rightarrow w[i]) \in R\} & \text{if } i = j, \\ \{X : (X \rightarrow AB) \in R \wedge \exists k : A \in M[i, k-1] \wedge B \in M[k, j]\} & \text{if } i < j. \end{cases}$$

Algorithm 3 illustrates how to compute M . Since each position in the array is a set of non-final symbols, we must choose an appropriate data structure for this. This can be done by a bit vector with an entry for each non-final symbol. Thus we see that the time complexity is of order $O(n^3m)$ and the memory usage is bounded by $O(n^2m)$, as m is an upper bound on the number of non-final symbols.

Algorithm 3 Given a set final symbols T , a set of non-final symbols N , the initial symbol S , a set of rules R and a word w of length n , this algorithm returns 1 if the word w of length n can be derived from S by the rules in R .

```

1: function GRAMMAR( $T, N, S, R, w, n$ )
2:    $M \leftarrow$  an  $n \times n$  dimensional matrix
3:    $M[i, j] \leftarrow \{\}$  for all  $1 \leq i, j \leq n$   $\triangleright M[i, j]$  is a set of non-final symbols, initially empty.
4:   for  $i \leftarrow 1, \dots, n$  do
5:     for each rule  $(X \rightarrow w[i]) \in R$  do
6:       add  $X$  to  $M[i, i]$ 
7:   for  $\ell \leftarrow 2, \dots, n$  do
8:     for  $i \leftarrow 1, \dots, n - \ell + 1$  do
9:        $j \leftarrow i + \ell - 1$ 
10:      for  $k \leftarrow i + 1, \dots, j$  do
11:        for each rule  $(X \rightarrow AB) \in R$  do
12:          if  $A \in M[i, k-1]$  and  $B \in M[k, j]$  then
13:            add  $X$  to  $M[i, j]$ 
14:   return 1 if  $S \in M[1, n]$  else 0

```
